

AI Orchestration Service

FastAPI Backend Implementation Documentation

1. Project Overview

A FastAPI-based AI Orchestration Backend for a legal-tech microservices system. The service handles request routing, validation, and workflow orchestration without implementing AI logic directly. Built using Clean Architecture and Hexagonal patterns.

2. Technology Stack

Component	Technology
Framework	FastAPI 0.109+
Validation	Pydantic 2.6+
Configuration	pydantic-settings 2.1+
Testing	pytest 9.0+, pytest-asyncio, httpx
Logging	Structured JSON with request ID
Python	3.13+

3. Architecture

The project follows Clean Architecture with 4 layers and Hexagonal (Ports & Adapters) pattern:

Layer	Purpose	Location
API	HTTP endpoints, schemas, middleware	app/api/
Core	Domain models, orchestrator, validators, workflow logic	app/core/
Interfaces	Abstract ports for external services	app/interfaces/
Infrastructure	Adapters, config, logging	app/infrastructure/

4. Implementation Steps

#	Step	Description	Status
1	FastAPI Skeleton	Project structure, main app, health endpoints	✓
2	API Schemas	Pydantic request/response models	✓
3	Domain Models	Entities, enums, value objects	✓
4	Task Classification	Explicit task type routing	✓

5	Business Validation	Payload validation per task type	✓
6	Task Orchestrator	Central coordination component	✓
7	Workflow Registry	Dynamic workflow management	✓
8	Workflow Interface	Abstract base for workflows	✓
9	Stubbed Workflows	5 workflow implementations	✓
10	AI Service Interfaces	LLM, RAG, Prompt ports	✓
11	Infrastructure Adapters	Stub service implementations	✓
12	Structured Logging	JSON logging with request ID	✓
13	Testing Strategy	Unit + integration tests	✓

5. Supported Task Types

Task Type	Description
LEGAL_CHAT	Interactive legal Q&A conversations
DOCUMENT_GENERATION	Generate legal documents from templates
CONTRACT_ANALYSIS	Analyze contract clauses and terms
CONTRACT_REFRAMING	Rewrite clauses from different perspectives
CASE_EVALUATION	Evaluate legal cases and provide assessments

6. Supported Jurisdictions

- Egypt (EG) - Arabic and English
- UAE (AE) - Arabic and English

7. AI Service Interfaces (Ports)

Interface	Purpose	Key Methods
LLMServiceInterface	Language model operations	generate(), generate_with_context()
RAGServiceInterface	Vector search & retrieval	retrieve(), search_similar()
PromptServiceInterface	Template management	get_template(), assemble_prompt()

8. Test Coverage

Test Suite	Count	Scope
Unit: TaskClassifier	7	Task type classification logic
Unit: BusinessValidator	13	Payload validation rules
Unit: WorkflowRegistry	7	Workflow registration/retrieval

Integration: API	14	Full request/response cycle
Total	41	All tests passing ✓

9. API Endpoints

Method	Endpoint	Description
GET	/health	Basic health check
GET	/ready	Readiness probe
POST	/api/v1/tasks	Execute AI task

10. Running the Service

Start the development server:

```
uvicorn app.api.main:app --reload
```

Run tests:

```
pytest tests/ -v
```